

An Efficient Vibe Coding Process for AI-Assisted Software Development

Vu Thi Thanh Huong¹, Quan Do Minh² and Nguyen Thi Ngoc Thanh³

¹ University of Finance – Marketing, ² Saigon University, ³ Ho Chi Minh City Open University
thanh.ntn@ou.edu.vn

Abstract. In recent years, the rapid advancement of AI agents has accelerated the adoption of AI-driven software development, where users can express ideas in natural language and iteratively refine generated outputs. Despite this convenience, such practices often result in unclear requirements, limited reusability, and low maintainability. This paper presents a controlled vibe coding process designed to support rapid software development while preserving maintainability and testability. As part of the evaluation, the study compares the testability of two code structures: a non-decoupled version and a decoupled version. The results show that the non-decoupled version produced 3 failed tests, whereas the decoupled version provided a clearer and more stable structure for unit and integration testing. In addition, the study compares PRD v1.0 and PRD v2.0 to examine the maturity of requirements artifacts, and includes an estimated comparison of development time with and without vibe coding across the studied workflow. The results suggest that PRD v2.0 provides clearer business flows and component boundaries, while the workflow with vibe coding reduces the total estimated development time from 78–93 hours to 44–51 hours. These findings suggest that AI-assisted development becomes more effective in practice when it is supported by a clearer process, stronger requirements artifacts and a more modular code structure.

Keywords: vibe coding, software development, human-in-the-loop, decoupling, software testing.

1 Introduction

The rapid growth of AI agents has strongly driven the rise of AI-supported software development. In this context, vibe coding can be viewed as a method for users to describe their ideas in natural language and quickly turn those ideas into working software [1]. Recent review studies show that AI agents are increasingly used across software development tasks, including code generation, documentation, requirements analysis, debugging, and maintenance [2]. AI programming tools such as GitHub Copilot have also shown their potential to improve development speed in a period of fast technological change [3].

However, a purely prompt-to-code approach still reveals several limitations when moving from rapid prototyping to deliverable software development. In practice, AI-generated code may achieve high speed in implementing initial functionality, but it often lacks requirement traceability, clear boundaries between user interface and business

logic, and readiness for systematic testing. Despite the growing adoption of AI-assisted development tools, existing studies often focus on isolated stages of the software lifecycle, such as code generation, testing, or requirements engineering. These approaches lack an integrated workflow that connects rapid prototyping with maintainable and testable software systems. To address this gap, this paper proposes a structured vibe coding process consisting of seven stages: business idea formulation, low-code specification, PRD construction, proof-of-concept (POC) development, decoupling, testing, and GitHub delivery. The main contributions of this paper are as follows: (i) proposing a structured vibe coding process from idea formation to code delivery; (ii) clarifying the role of the human-in-the-loop mechanism in refining specification documents; (iii) showing that the decoupling step can better support unit and integration testing; and (iv) providing an estimated comparison of development time with and without vibe coding across the studied workflow.

2 Related works

Table 1. Representative studies related to AI-supported software development.

Research	Contributions
Takerngsaksiri et al. (2024) [4]	Proposes a human-in-the-loop framework for software development agents, showing that developer feedback can guide planning and code generation, although code quality still requires careful review.
Wang et al. (2023) [5]	Reviews 102 studies on the use of LLMs in software testing, with test case preparation and program repair as the most common directions.
Garousi et al. (2019) [6]	Reviews 208 studies on software testability and identifies observability and controllability as the most important factors.
Ramesh and Jarke (2001) [7]	Shows that traceability is important for maintaining links between requirements and software development outputs.
Cordeiro et al. (2024) [8]	An empirical study on 30 Java projects showing that LLMs can support refactoring, although they still perform worse than humans in context-dependent cases.
Zadenoori et al. (2025) [9]	Reviews 74 studies on LLMs for Requirements Engineering; most focus on elicitation and validation in controlled settings.

Recent work has examined several parts of AI-supported software development. Table 1 summarizes representative studies related to human-in-the-loop development agents, software testing, software testability, requirements traceability, code refactoring, and requirements engineering. Takerngsaksiri et al. [4] introduced a human-in-the-loop framework for software development agents, in which engineers can review and guide coding plans and generated code during the development process. Their study suggests that such human involvement can reduce development time and effort for straightforward tasks, although code quality remains a concern in some cases. Wang et al. [5] reviewed the use of large language models in software testing, while Garousi et

al. [6] discussed the main factors that affect software testability. Ramesh and Jarke [7] highlighted the importance of traceability between requirements and downstream artifacts. Cordeiro et al. [8] examined the refactoring capability of large language models, and Zadenoori et al. [9] reviewed their use in requirements engineering.

These studies cover important stages of the software lifecycle, but they are often treated separately. In contrast, this study integrates these aspects into a more connected vibe coding process that links requirements refinement, implementation, and testing.

3 Methodology

The methodology is based on a software development process. It begins with defining the business idea and the main system scope. The process then continues with low-code specification, PRD construction and refinement, POC development, decoupling for maintainability and testability, unit and integration testing, and GitHub delivery. These stages are connected and support the transition from an initial idea to a more structured, testable, and deliverable software product.

3.1 Workflow Overview

The workflow of the proposed system is illustrated in Fig. 1. This diagram demonstrates how an initial business idea is progressively transformed into a more structured and testable software artifact through a sequence of connected stages. The process starts with business idea formation and low-code specification, which are then translated into a Product Requirements Document (PRD). A human-in-the-loop step is introduced to refine the PRD before implementation. Based on the refined requirements, proof of concept (POC) is developed to validate the main business flows. The resulting code is refined through decoupling to improve modularity and reduce unnecessary dependencies, thereby creating a stronger basis for unit and integration testing. Finally, the workflow is completed through GitHub, which supports version control, collaboration, and code review.

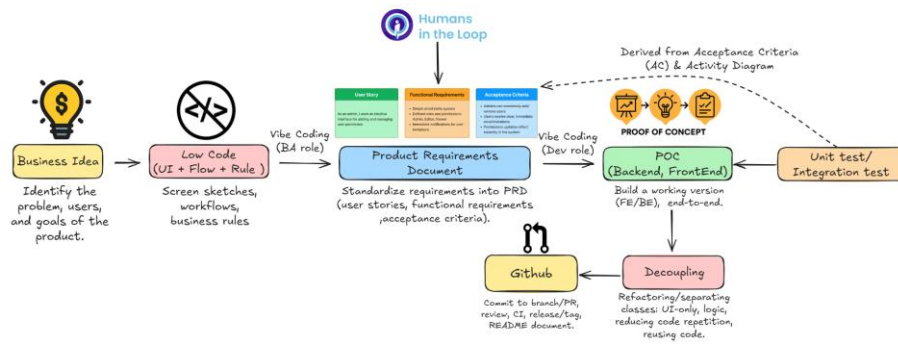


Fig. 1. Overall workflow of the proposed method for controlled vibe coding in software development.

3.2 Requirements Structuring and Prototype Development

This stage focuses on transforming the initial business idea into increasingly structured development artifacts. The process begins with the identification of the MVP scope, core user flows, and main functional modules. These ideas are then expressed through low-code specifications to clarify screens, workflows, business rules, and expected interactions. PRD v1.0 is used to support rapid implementation, while PRD v2.0 is refined through human review to check and improve. Using PRD v1.0 as the main implementation reference, a proof of concept (POC) is then developed to validate the core business flows of the selected modules.

Business Idea. The process begins with the identification of the minimum viable product (MVP) scope and the main functional modules to be implemented. In this study, the selected case is an admin dashboard consisting of three core modules: Admin Login, Manage POIs, and Manage Tours, as shown in Fig. 2. The first step is the main objective to define what should be built and which business flows are essential for validating the idea.

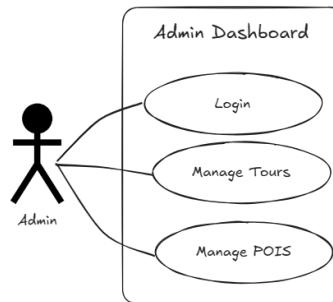


Fig. 2. Modules scope of the admin dashboard covering login, tour management, and POI management.

Low Code specification. After the business idea, the main screens, user flows are expressed in a low-code form. This step will help clarify the system behavior before writing the PRD and implementation. In this study, the low code specification was supported by Google AI Studio to outline the screens and interaction flows of Login, Manage POIs and Tours modules. As illustrated in Fig. 3, it helped quickly transform the business description into an initial interface and functional flow draft before constructing the PRD and POC.

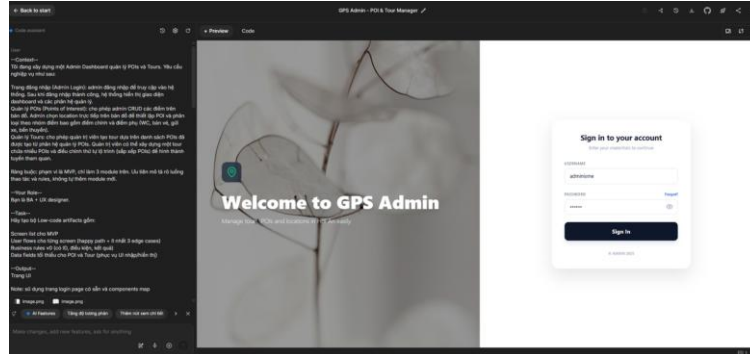


Fig. 3. Google AI Studio in the low-code specification stage, showing the generation of an initial screen from business requirements.

PRD construction and refinement. After the requirements have been transformed into low-code artifacts, the next step is to construct a Product Requirements Document (PRD) to standardize this information into a more structured requirements document. In this study, Claude was used as the primary AI tool to support PRD construction. Based on the outputs of the low-code specification stage, such as screen lists, user flows, happy paths, edge cases, business rules, and reuse notes, Claude was used to reorganize the content into more structured elements, including product goals, MVP scope, feature lists, user stories, functional requirements, acceptance criteria, required input data, integration assumptions, and implementation constraints. The PRD is developed at two levels. PRD v1.0 is prepared to support the implementation, while PRD v2.0 is refined through human review to verify and improve testing.

POC development. Based on PRD v1.0, a proof of concept (POC) is developed to validate the business flows of modules. In this study, Visual Studio Code and GitHub Copilot were used to support code generation. The main objective of this stage is to produce a runnable version of the system with the defined scope. Since the backend was already available, this stage focused only on front-end development. Because rapid implementation was prioritized, the resulting code still has tight coupling among interface, business logic and API. This problem will be solved in the next stage through the decoupling.

3.3 Structural Refinement and Testing

Decoupling for Maintainability and Testability. This stage focuses on clarifying the boundaries of responsibility among system components, transforming a runnable prototype into a structured codebase. In this study, decoupling was carried out using Feature-Sliced Design (FSD), with the source code reorganized into clear functional layers such as app, pages, widgets, features, entities, and shared, as illustrated in Fig. 4. This structure helps reduce cross dependencies, improve reusability, and provide a stronger foundation for maintainability and testing.

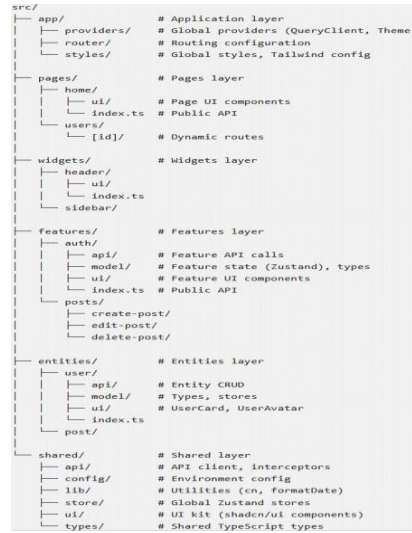


Fig. 4. FSD-oriented codebase structure after decoupling.

Unit and Integration Testing. After the codebase is restructured through the decoupling stage, the process moves to the construction of unit tests and integration tests to verify the correctness of the system at different levels. If decoupling clarifies the boundaries among the user interface, business logic, APIs, and shared components, this stage takes advantage of that structure to organize testing in a more systematic way. In this study, testing was carried out using Vitest, which supports fast test execution and is well suited to the refactored frontend codebase. As illustrated in Fig. 5, the test code is organized following the Feature-Sliced Design approach, with test files placed close to their corresponding modules and shared testing utilities separated into the test/ directory.

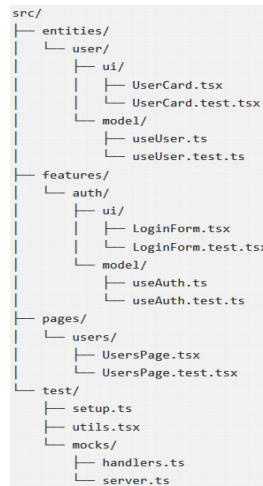


Fig. 5. FSD-based test organization used for Vitest-based unit and integration testing.

Github Delivery. The last stage focuses on delivering the project through Github. In this study, we used Github to manage branches, commits and review pull requests. This helps organize the development process and supports collaboration among team members.

4 Implementation and Evaluation

4.1 Case study context and evaluation

This study evaluates the proposed workflow through an admin dashboard case study with three modules: Login, Manage POIs, and Manage Tours. The case combines interface interactions, business rules, and API-connected front-end flows. Since the backend was already available, the implementation focuses on front-end development. The workflow uses Google AI Studio for low-code specification, Claude for PRD construction, GitHub Copilot with Visual Studio Code for POC development, Feature-Sliced Design for decoupling, and Vitest for testing.

The evaluation focuses on three comparisons. First, requirements maturity is examined through the transition from PRD v1.0 to PRD v2.0, particularly the addition of activity and component diagrams. Second, the codebase is compared before and after decoupling under the same testing scope. Third, the study provides an estimated comparison between the proposed workflow and a conventional development process across the entire workflow.

4.2 From PRD v1.0 to PRD v2.0

The first result of the proposed process is reflected in the transition from PRD v1.0 to PRD v2.0. As shown in Table 1, the two versions retain the same MVP functional scope, including three main modules, 18 user stories, and 18 acceptance criteria. This indicates that PRD v2.0 is not intended to expand the system functionally, but rather to improve the clarity and usability of the requirements document for the subsequent development stages.

Table 1. Comparison between PRD v1.0 and PRD v2.0.

Criterion	PRD v1.0	PRD v2.0
MVP modules	3	3
User stories	18	18
Acceptance criteria	18	18
Activity diagrams	No	Yes
Component diagram	No	Yes
UI/POC generation	High	High
Unit testing & integration testing support	Limited	Higher
FSD alignment	Medium	High

The main difference between the two versions lies in the maturity of the specification artifacts. While PRD v1.0 mainly supports rapid UI and POC development, PRD v2.0 is refined through a human-in-the-loop process to clarify business flows, component boundaries, and test readiness. As illustrated in Fig. 6, PRD v2.0 adds three activity diagrams for Authentication, POI Management, and Tour Management, together with a component diagram to clarify the system structure.

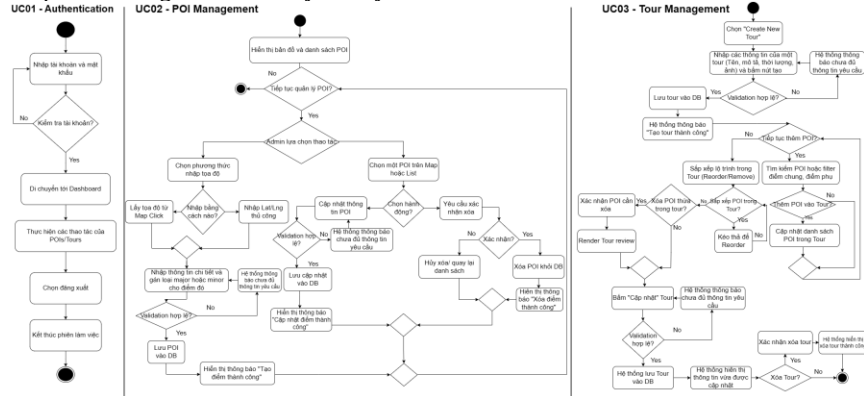


Fig. 6. Activity diagrams added in PRD v2.0 for the three main use cases: authentication, POI management, and tour management.

4.3 Comparing unit tests and integration tests before and after decoupling

As shown in Table 2, the pre-decoupling codebase based on PRD v1.0 and the post-decoupling codebase based on PRD v2.0 have now reached the same testing scale, with 231 unit tests, 79 integration tests, 1 infrastructure test, and 311 tests in total. This indicates that the difference between the two versions no longer lies in the number of tests, but mainly in execution stability and the extent to which the system architecture supports testing.

Table 2. Comparison of unit and integration testing before and after decoupling

Criterion	Before decoupling + PRD v1.0	After decoupling + PRD v2.0
Unit tests (all)	231	231
Integration tests (all)	79	79
Infrastructure tests	1	1
Total tests	311	311
Runner result	308 passed / 3 failed	311 passed / 0 failed

In the pre-decoupling version, the test results recorded 308 passed tests and 3 failed tests, mainly in API hooks when verifying error-state transitions after rejected API calls. This indicates that the limitation did not lie in the number of test cases, but in the stability of the hook logic and the error-handling behavior at the API layer.

In contrast, in the post-decoupling version, the codebase was reorganized more clearly and supported by PRD v2.0 with more detailed acceptance criteria and diagrams. As a result, the same testing scope achieved more stable outcomes, with 311 out of 311 tests passing. This result suggests that the value of decoupling and PRD v2.0 lies in improving systematic testability rather than merely increasing the number of tests.

4.4 Comparison between development with and without vibe coding

This study also includes an approximate comparison between the proposed vibe coding workflow and a comparable frontend development process without vibe coding. In the present case, the vibe coding workflow was completed in approximately 48 hours. A conventional workflow for the same scope is estimated to be slower because requirement structuring, UI drafting, PRD preparation and initial implementation involve more manual work. Since this comparison was not conducted as a controlled time-tracking experiment, the values in Table 3 should be interpreted as indicative estimates rather than formal productivity measurements.

Table 3. Comparison of development stages with and without vibe coding.

Stages	With Vibe Coding	Without Vibe Coding
Business scope clarification	4–5 hours	8–10 hours
Low-code	3–4 hours	8–10 hours
PRD construction	5–6 hours	16–18 hours
Initial frontend implementation	16–18 hours	24–28 hours
Decoupling and testing preparation	14–15 hours	20–24 hours
GitHub delivery	2–3 hours	2–3 hours
Total	44–51 hours	78–93 hours

Table 3 shows an estimated comparison of development time with and without vibe coding across the studied workflow. The workflow with vibe coding takes less time in most stages, particularly in scope clarification, low-code drafting, PRD construction, and frontend implementation. The time difference becomes smaller in the decoupling and testing stage, and GitHub delivery remains almost the same in both cases. In total, the process with vibe coding is estimated at 44–51 hours, while the process without vibe coding is estimated at 78–93 hours. These results suggest that vibe coding can reduce the overall development time for the studied workflow.

5 Discussion and Conclusion

5.1 Discussion

These results show that the main value of the proposed process is not limited to faster development. Its contribution also appears in the quality of the software artifacts

produced in later stages. PRD v2.0 does not add new functions compared with PRD v1.0, but it makes business flows, acceptance criteria, and component boundaries clearer with activity diagrams and component diagrams. This provides a better way for decoupling and testing. These findings suggest that the value of vibe coding should be considered not only in terms of implementation speed, but also in terms of how well the resulting software can be structured, tested.

5.2 Conclusion

In this study, we introduced a structured vibe coding process that combines the speed of AI-assisted development with the practical needs of testing, maintainability, and software delivery. Through a case study of an admin dashboard with three modules, namely Admin Login, Manage POIs, and Manage Tours, the results show that the proposed process supports the transition from an AI-generated proof of concept to a more structured and testable codebase.

Acknowledgments. The authors sincerely thank their supervisors and colleagues for their support and constructive feedback.

Disclosure of Interests. The authors declare no competing interests.

References

1. Sarkar, A., Drosos, I.: Vibe coding: programming through conversation with artificial intelligence. arXiv preprint arXiv:2506.23253 (2025).
2. Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H.: Large Language Models for Software Engineering: A Systematic Literature Review. arXiv preprint arXiv:2308.10620 (2023).
3. Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M.: The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590 (2023).
4. Takeringsaksiri, W., Pasuksmit, J., Thongtanunam, P., Tantithamthavorn, C., Zhang, R., Jiang, F., Li, J., Cook, E., Chen, K., Wu, M.: Human-In-the-Loop Software Development Agents. arXiv preprint arXiv:2411.12924 (2024).
5. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software Testing with Large Language Models: Survey, Landscape, and Vision. arXiv preprint arXiv:2307.07221 (2023).
6. Garousi, V., Felderer, M., Kılıçaslan, F.N.: A Survey on Software Testability. *Information and Software Technology* 108, 35–64 (2019).
7. Ramesh, B., Jarke, M.: Toward Reference Models for Requirements Traceability. *IEEE Transactions on Software Engineering* 27(1), 58–93 (2001).
8. Cordeiro, J., Noei, S., Zou, Y.: An Empirical Study on the Code Refactoring Capability of Large Language Models. arXiv preprint arXiv:2411.02320 (2024).
9. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv preprint arXiv:2509.11446 (2025).